

Universidad Tecnológica de Pereira

Facultad de Ingenierías

Programa de Ingeniería de Sistemas y Computación

HPC

Implementación del Método de Jacobi para resolución de la Ecuación de Poisson 1D

Presentado por:

Juan Esteban Cardona Blandón

Jhoan Esteban Corrales Rodríguez

Sebastian Perdomo

Juana Valentina Martínez

Profesor:

Ramiro Andrés Barrios Valencia

Pereira, Colombia

26 de marzo de 2026

Índice

1. Marco Teórico	2
1.1. Computación de Alto Rendimiento (HPC)	2
1.2. La Ecuación de Poisson en 1D	2
1.3. Discretización del Operador de Poisson	3
1.4. El Sistema Lineal Discretizado	3
1.5. El Método Iterativo de Jacobi	4
1.6. Convergencia del Método de Jacobi	4
1.7. Programación Secuencial y Concurrente	5
1.8. Hilos y la Librería Pthreads	5
1.9. Procesos y la Llamada al Sistema Fork	6
1.10. Jerarquía de Caché y Localidad de Memoria	6
1.11. Métricas de Rendimiento en Computación Paralela	7
2. Marco Conceptual	8
2.1. Características del PC	8
2.2. Características del software	8
3. Desarrollo de la implementación	8
3.1. Problema numérico base	8
3.2. Arquitectura del código	9
3.3. Arquitectura general de datos	10
3.4. Implementación secuencial estándar	10
3.5. Implementación secuencial con alineación a línea de caché	11
3.6. Implementación paralela con hilos	11
3.7. Implementación paralela con procesos (fork)	12
4. Pruebas	14
4.1. Resultados del algoritmo secuencial	14
4.2. Resultados del algoritmo secuencial optimizado por cache	15
4.3. Resultados de las pruebas del algoritmo optimizado por compilador	17
4.4. Resultados del algoritmo paralelizado con hilos	20
4.5. Resultados del algoritmo paralelizado por procesos	24
5. Comparaciones	29
6. Conclusiones	31

1. Marco Teórico

1.1. Computación de Alto Rendimiento (HPC)

La Computación de Alto Rendimiento (HPC, por sus siglas en inglés) consiste en la agregación de potencia de cálculo a través de arquitecturas paralelas para resolver problemas científicos y tecnológicos de extrema complejidad (Hager & Wellein, 2010). Esta disciplina permite ejecutar algoritmos y modelados matemáticos que resultarían inabarcables para un sistema tradicional debido a restricciones de tiempo y recursos de hardware. Al dividir cargas masivas de trabajo en instrucciones más pequeñas que se procesan simultáneamente, HPC maximiza la velocidad de procesamiento de datos y la obtención de resultados en la ingeniería (Patterson & Hennessy, 2017).

Históricamente, el aumento de la frecuencia de reloj en los microprocesadores permitía acelerar la ejecución de algoritmos secuenciales de forma directa. Sin embargo, tras alcanzar los límites físicos y térmicos del silicio, la industria se desplazó hacia arquitecturas multinúcleo que exigen un paradigma de programación paralela (Patterson & Hennessy, 2017). Bajo este enfoque, el diseño de software eficiente requiere distribuir la carga de trabajo de manera concurrente, haciendo del análisis de algoritmos iterativos numéricos, como el método de Jacobi, un caso de estudio en HPC.

1.2. La Ecuación de Poisson en 1D

La ecuación de Poisson en una dimensión es una ecuación diferencial de segundo orden de la forma:

$$-\frac{d^2u}{dx^2} = f(x) \quad (1)$$

definida sobre un dominio cerrado $[a, b]$, donde $f(x) \in C[a, b]$ es una función de forzamiento dada (Burkardt, 2011). Para garantizar la unicidad de la solución, esta ecuación se complementa con condiciones de frontera de Dirichlet en los extremos del dominio:

$$u(a) = u_a, \quad u(b) = u_b \quad (2)$$

Un ejemplo concreto y representativo se define sobre el intervalo $[0, 1]$ con condiciones $u(0) = u(1) = 0$, función de forzamiento $f(x) = -x(x + 3)e^x$ y solución analítica exacta $u(x) = x(x - 1)e^x$.

La ecuación de Poisson es la representación estacionaria de la ecuación de calor y aparece de forma ubícua en simulación de fluidos, electrostática, mecánica de sólidos y transferencia de calor (Elman et al., 2005). Su relevancia en HPC radica en que su resolución numérica,

especialmente para dominios de alta dimensión o mallas finas, genera sistemas lineales de gran escala cuyo costo computacional motiva el uso de métodos iterativos y estrategias de paralelismo (Burkardt, 2011).

1.3. Discretización del Operador de Poisson

La solución numérica de la ecuación de Poisson requiere reemplazar el operador diferencial continuo por una aproximación discreta sobre una malla de N nodos x_j distribuidos uniformemente en $[a, b]$ con espaciado $h = (b - a)/(N - 1)$ (Burkardt, 2011). Aplicando diferencias finitas centradas, la segunda derivada en el nodo x_j se aproxima mediante el cociente de diferencias:

$$-\frac{d^2u}{dx^2}(x_j) \approx \frac{-u_{j-1} + 2u_j - u_{j+1}}{h^2} \quad (3)$$

Esta expresión es una aproximación de segundo orden al operador de Poisson y constituye la ecuación discreta en cada nodo interior de la malla (Burkardt, 2011). Para garantizar que los errores de discretización disminuyan apropiadamente al refinar la malla, se selecciona una secuencia anidada de grillas con $n_k = 2^k + 1$ nodos, de forma que el espaciado $h_k = (b - a)/2^k$ converge a cero conforme $k \rightarrow \infty$.

1.4. El Sistema Lineal Discretizado

Al aplicar las ecuaciones de diferencias finitas en todos los nodos interiores, junto con las condiciones de frontera de Dirichlet, se obtiene un sistema lineal $A \cdot \mathbf{u} = \mathbf{f}$ de dimensión $N \times N$ (Burkardt, 2011). La matriz del sistema $h^2 A$ es tridiagonal, simétrica y definida positiva, con la estructura:

$$h^2 A = \begin{pmatrix} 1 & 0 & 0 & 0 & \cdots & 0 \\ -1 & 2 & -1 & 0 & \cdots & 0 \\ 0 & -1 & 2 & -1 & \cdots & 0 \\ \vdots & & \ddots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & -1 & 2 & -1 \\ 0 & 0 & \cdots & 0 & 0 & 1 \end{pmatrix} \quad (4)$$

Las propiedades de simetría y definición positiva son condiciones suficientes para garantizar la convergencia de los métodos iterativos clásicos, en particular la iteración de Jacobi (Burkardt, 2011). Este sistema presenta una estructura dispersa (*sparse*) altamente favorable desde el punto de vista computacional, ya que cada fila contiene a lo sumo tres elementos no nulos, lo que reduce significativamente el costo de memoria y operaciones por iteración (Barrett et al., 1994).

1.5. El Método Iterativo de Jacobi

Dado un sistema lineal $A \cdot \mathbf{x} = \mathbf{f}$ y una aproximación inicial $\mathbf{x}^{(0)}$, la iteración de Jacobi genera una secuencia de aproximaciones $\mathbf{x}^{(0)}, \mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots$ mediante la actualización simultánea de todos los componentes de la solución (Burkardt, 2011). La actualización del i -ésimo componente en la iteración $(k + 1)$ se define como:

$$x_i^{(k+1)} = \frac{f_i - \sum_{j \neq i} A_{ij} \cdot x_j^{(k)}}{A_{ii}} \quad (5)$$

Bajo la descomposición matricial $A = L + D + U$, donde L , D y U son las partes triangular inferior, diagonal y triangular superior respectivamente, la iteración se expresa de forma vectorizada como:

$$\mathbf{x}^{(k+1)} = D^{-1}(\mathbf{f} - (L + U) \mathbf{x}^{(k)}) \quad (6)$$

Esta formulación vectorizada es computacionalmente eficiente, pues la actualización de cada componente en una iteración es completamente independiente de los demás, propiedad que resulta fundamental para explotar el paralelismo de datos mediante hilos y procesos (Burkardt, 2011).

1.6. Convergencia del Método de Jacobi

El criterio de parada de la iteración de Jacobi se basa en el monitoreo del residual $\mathbf{r}^{(k)} = A \cdot \mathbf{x}^{(k)} - \mathbf{f}$ (Burkardt, 2011). La métrica estándar empleada es la norma RMS del residual:

$$\text{Residual RMS} = \frac{\|\mathbf{r}^{(k)}\|}{\sqrt{N}} \leq \varepsilon \quad (7)$$

que permite comparar la convergencia entre sistemas de diferentes tamaños de malla con una tolerancia fija ε (Burkardt, 2011). Un aspecto crítico del método de Jacobi aplicado al problema de Poisson es su convergencia lenta: el número de iteraciones m_k requeridas crece aproximadamente como $m_k \sim \mathcal{O}(n_k^2)$, implicando que el costo computacional total escala como $\mathcal{O}(n_k^3)$. Esta degradación severa al refinar la malla motiva directamente la necesidad de paralelismo: al distribuir las iteraciones entre múltiples hilos o procesos, es posible reducir el tiempo de ejecución real incluso cuando el número total de operaciones aritméticas permanece constante (Briggs et al., 2000).

1.7. Programación Secuencial y Concurrente

La programación secuencial es el paradigma clásico donde las instrucciones de un algoritmo se ejecutan de forma estrictamente lineal sobre un único núcleo de procesamiento (Ben-Ari, 2006). En este modelo, el límite de rendimiento está directamente dictado por la frecuencia de reloj del hardware y el número de iteraciones del algoritmo. Como respuesta a estas limitaciones, la programación concurrente estructura el software en múltiples hilos o tareas independientes que hacen progreso de forma superpuesta en el tiempo, reduciendo drásticamente el tiempo de ejecución en problemas de cómputo intensivo.

Para el algoritmo de Jacobi, la paralelización es conceptualmente directa: dado que la actualización de cada nodo $x_i^{(k+1)}$ depende únicamente de los valores de la iteración anterior $\mathbf{x}^{(k)}$, el cálculo de todos los nodos interiores en una misma iteración es completamente independiente entre sí (Burkardt, 2011). Esta independencia de datos por iteración elimina las condiciones de carrera durante la fase de actualización, permitiendo distribuir los nodos entre hilos o procesos sin necesidad de sincronización intra-iteración, aunque sí se requiere una barrera de sincronización entre iteraciones consecutivas (Silberschatz et al., 2018).

1.8. Hilos y la Librería Pthreads

En el ámbito de la computación paralela, el modelo de memoria compartida establece que múltiples hilos de ejecución creados por la librería POSIX Threads (Pthreads) operan dentro del mismo espacio de direccionamiento lógico del proceso padre (Silberschatz et al., 2018). A nivel de arquitectura, esto significa que todos los núcleos del procesador acceden de forma directa y simultánea a las mismas estructuras de datos residentes en la memoria principal, sin necesidad de copiar o transmitir datos entre unidades de procesamiento. Para el algoritmo de Jacobi, este modelo es altamente eficiente, pues el vector de solución $\mathbf{u}$ y el vector de forzamiento $\mathbf{f}$ se comparten íntegramente entre todos los hilos, y cada uno opera únicamente sobre un subconjunto disjunto de nodos, eliminando la contención de escritura (Burkardt, 2011).

La creación de un hilo mediante `pthread_create` y su sincronización mediante `pthread_join` introduce una sobrecarga fija por hilo que resulta despreciable cuando el número de nodos por hilo es suficientemente grande (Silberschatz et al., 2018). No obstante, entre iteraciones de Jacobi es imprescindible garantizar que todos los hilos hayan completado la actualización de sus nodos antes de iniciar la siguiente iteración; esto se logra mediante una barrera de sincronización (`pthread_barrier_wait`), cuyo costo adicional es un factor determinante en el análisis de escalabilidad (Hager & Wellein, 2010).

1.9. Procesos y la Llamada al Sistema Fork

La paralelización mediante la llamada al sistema `fork()` crea procesos hijos independientes que no comparten espacio de memoria con el proceso padre, operando bajo el modelo de memoria distribuida dentro de un mismo nodo físico (Silberschatz et al., 2018). Cada proceso hijo posee su propia copia privada del espacio de direcciones, obtenida mediante el mecanismo de *copy-on-write* del sistema operativo, lo que elimina las condiciones de carrera pero introduce una sobrecarga significativa en la creación, gestión y comunicación entre procesos. Para el algoritmo de Jacobi, cada proceso hijo calcula la actualización de un subconjunto de nodos, pero los resultados parciales deben comunicarse al proceso padre mediante mecanismos de IPC (*Inter-Process Communication*) como tuberías (*pipes*) o memoria compartida explícita (`shmget/mmap`) antes de iniciar la siguiente iteración.

La principal desventaja del modelo basado en `fork` para algoritmos iterativos como Jacobi reside en el costo acumulado de esta comunicación inter-iteración (Silberschatz et al., 2018). Dado que el método de Jacobi puede requerir miles de iteraciones para converger, especialmente en mallas finas, la sobrecarga de sincronizar resultados entre procesos en cada iteración puede superar ampliamente la ganancia obtenida por la distribución del cómputo, comportamiento análogo al evidenciado previamente en la paralelización por procesos de la multiplicación de matrices (Hager & Wellein, 2010).

1.10. Jerarquía de Caché y Localidad de Memoria

El rendimiento de una aplicación HPC no depende exclusivamente de la capacidad matemática del procesador, sino también del acceso al subsistema de memoria (Drepper, 2007). Los procesadores modernos integran una jerarquía de memoria ultrarrápida (L1, L2 y L3) cuya unidad mínima de transferencia es la *cache line* de 64 bytes. Para el algoritmo de Jacobi en 1D, el patrón de acceso al vector de solución $\mathbf{u}$ es estrictamente secuencial (nodo $j - 1$, j , $j + 1$), lo que favorece de forma natural la localidad espacial de memoria y maximiza el aprovechamiento de la caché, a diferencia de los accesos por columnas del caso matricial (Patterson & Hennessy, 2017).

Sin embargo, en la implementación paralela, la división del vector entre múltiples hilos introduce el fenómeno de *false sharing*: si dos hilos adyacentes operan sobre nodos que residen en la misma línea de caché, las escrituras de un hilo invalidan la línea en el caché del otro, generando una contención invisible a nivel de código que penaliza severamente el rendimiento. Garantizar que cada hilo opere sobre bloques de nodos suficientemente grandes, y alineados con el tamaño de la línea de caché, es una condición necesaria para minimizar este efecto en la implementación paralela del método de Jacobi (Drepper, 2007).

1.11. Métricas de Rendimiento en Computación Paralela

Para cuantificar el éxito de una implementación paralela, la literatura científica emplea métricas matemáticas estandarizadas (Hager & Wellein, 2010). La métrica fundamental es el *Speedup* o Aceleración S_p , definido como:

$$S_p = \frac{T_1}{T_p} \quad (8)$$

donde T_1 es el tiempo de ejecución del algoritmo secuencial óptimo y T_p es el tiempo paralelo con p unidades de procesamiento (Hager & Wellein, 2010). Derivada de esta métrica, la Eficiencia E_p evalúa el grado de aprovechamiento de los recursos físicos:

$$E_p = \frac{S_p}{p} = \frac{T_1}{p \cdot T_p} \quad (9)$$

El desempeño asintótico de estos sistemas está regido por la **Ley de Amdahl**, que establece que el Speedup máximo está limitado por la fracción no paralelizable ($1 - f$) del algoritmo:

$$S_{\text{Amdahl}} \leq \frac{1}{(1 - f) + \frac{f}{p}} \quad (10)$$

Para el algoritmo de Jacobi, la fracción secuencial incluye la evaluación del residual global de convergencia y la barrera de sincronización entre iteraciones, que no pueden paralelizarse y constituyen el cuello de botella principal según este modelo (Patterson & Hennessy, 2017). En contraste, la **Ley de Gustafson-Barsis** describe la escalabilidad débil y argumenta que, al incrementar el tamaño de la malla N , la fracción paralelizable del trabajo crece proporcionalmente, haciendo que la porción secuencial pierda peso relativo:

$$S_{\text{Gustafson}} = (1 - f) + p \cdot f \quad (11)$$

Bajo la visión de Gustafson, a medida que la malla se refina, la cantidad de operaciones paralelas aumenta drásticamente, mitigando el cuello de botella pronosticado por Amdahl y permitiendo un Speedup escalable con el número de unidades de procesamiento (Gustafson, 1988).

2. Marco Conceptual

2.1. Características del PC

- Procesador: Ryzen 5 7600X
- Cantidad de núcleos: 6
- Cantidad de subprocesos/hilos: 12
- Frecuencia básica del procesador: 4.7GHz
- Frecuencia Turbo máxima: 5.3GHz
- Memoria RAM: 32GB DDR5 @ 5200 MHz
- SSD: 1TB - R: 3500 MB/s - W: 1900 MB/s
- Sistema operativo: Arch Linux

2.2. Características del software

Se puede encontrar el código fuente del proyecto en el siguiente repositorio de GitHub:
<https://github.com/Juanes7222/HPC-Jacobi.git>

3. Desarrollo de la implementación

Esta sección explica cómo se construyó el solver de Poisson 1D con iteración de Jacobi, atendiendo tanto al resultado final como a las decisiones que lo determinaron. La idea central fue mantener una base matemática única y variar exclusivamente la estrategia computacional, de modo que las comparaciones de rendimiento sean atribuibles al mecanismo de ejecución y no a diferencias en el algoritmo.

3.1. Problema numérico base

El código resuelve la ecuación de Poisson 1D con condiciones de Dirichlet homogéneas:

$$-u''(x) = f(x), \quad x \in (0, 1), \quad u(0) = u(1) = 0. \quad (12)$$

El problema concreto utilizado en todas las pruebas es el definido por Burkardt (2011): la función forzante es

$$f(x) = -x(x + 3)e^x, \quad (13)$$

cuya solución analítica exacta es

$$u(x) = x(x - 1)e^x. \quad (14)$$

Disponer de solución exacta permite calcular el error en norma máxima al final de cada ejecución mediante `max_error`, lo que sirve para verificar que todas las variantes producen el mismo resultado numérico independientemente del modelo de paralelismo.

El dominio se discretiza con una malla uniforme de n puntos interiores con paso $h = 1/(n + 1)$. Aplicando diferencias finitas centradas en el punto i se obtiene:

$$\frac{-u_{i-1} + 2u_i - u_{i+1}}{h^2} = f_i. \quad (15)$$

Despejando u_i se obtiene la iteración de Jacobi usada en todas las variantes:

$$u_i^{(k+1)} = \frac{1}{2} \left(u_{i-1}^{(k)} + u_{i+1}^{(k)} + h^2 f_i \right). \quad (16)$$

La convergencia se verifica con el residual RMS calculado en `rms_residual`:

$$\text{RMS}(r) = \sqrt{\frac{1}{n} \sum_{i=1}^n r_i^2}, \quad r_i = \frac{-u_{i-1} + 2u_i - u_{i+1}}{h^2} - f_i. \quad (17)$$

Las iteraciones se detienen cuando $\text{RMS}(r) < 10^{-6}$ o al alcanzar el máximo configurado. Este criterio es el recomendado por Burkardt (2011, sección 9): medir la diferencia entre iterados sucesivos $\|u^{(k+1)} - u^{(k)}\|$ puede converger a cero mucho antes de que el sistema lineal quede satisfecho con la tolerancia deseada, por lo que no es una garantía confiable de calidad de solución.

3.2. Arquitectura del código

Antes de entrar en cada variante se fijaron tres principios que guiaron toda la implementación:

1. **Misma lógica numérica para todas las variantes:** si el algoritmo cambia, la comparación deja de ser limpia.
2. **Separación entre utilidades numéricas y motor de ejecución:** funciones compartidas en `poisson.c/poisson.h`; cada estrategia de ejecución en su propio archivo.

3. **Costo de sincronización visible:** diseñar las versiones paralelas de forma que el precio de cada modelo (barreras, semáforos) sea identificable estructuralmente.

Esta organización hace que cada archivo del directorio `src` tenga una responsabilidad única: `poisson.c` contiene las utilidades compartidas, y `serial_std.c`, `serial_cache.c`, `threads.c` y `processes.c` contienen cada uno una estrategia completa e independiente.

3.3. Arquitectura general de datos

Todas las implementaciones comparten la misma estructura de memoria de doble buffer:

- `u`: estado de la solución en la iteración k , usado solo para lectura.
- `u_new`: solución calculada en la iteración $k + 1$, usado solo para escritura.
- `f`: término fuente, calculado una sola vez antes del bucle principal.

El doble buffer es necesario para que la iteración de Jacobi sea correcta: cada `u_new[i]` depende exclusivamente de valores de `u` de la iteración anterior, nunca de valores ya actualizados en la misma pasada. Al final de cada iteración se intercambian los punteros sin copiar datos, lo que hace que el intercambio tenga costo $O(1)$ independientemente de n .

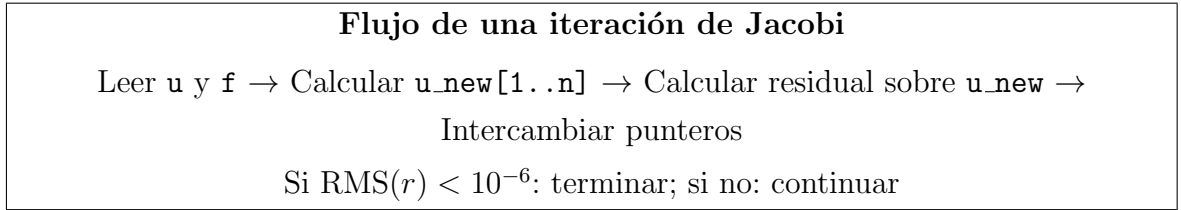


Figura 1: Ciclo general compartido por todas las variantes.

3.4. Implementación secuencial estándar

La versión secuencial estándar (`serial_std.c`) sirve como línea base para medir todas las demás. No aplica ninguna optimización explícita de memoria ni de compilador: los vectores `u` y `u_new` se reservan con `calloc`, que garantiza inicialización a cero, y `f` con `malloc`.

El bucle principal recorre $i = 1, \dots, n$, aplica la ecuación (16) sobre `u_new`, y a continuación evalúa el residual RMS sobre `u_new` antes del intercambio de punteros. El residual se evalúa sobre el nuevo iterado deliberadamente: de ese modo el criterio de parada refleja la calidad de la solución que se usará en la siguiente iteración, no la del estado anterior.

3.5. Implementación secuencial con alineación a línea de caché

Esta variante (`serial_cache.c`) optimiza la gestión de memoria sin alterar el algoritmo. Introduce dos cambios respecto a la versión estándar.

El primero es la reserva alineada: los tres vectores se asignan con `aligned_alloc` a 64 bytes, que es el tamaño de una línea de caché en x86-64. `malloc` estándar solo garantiza alineación a 8 bytes, lo que puede dejar el primer elemento de un arreglo partido entre dos líneas de caché, obligando al hardware a cargar dos líneas para acceder a un solo elemento. Con alineación a 64 bytes, `u[0]` cae siempre al inicio de una línea y el prefetcher del procesador puede cargar líneas completas desde el primer acceso.

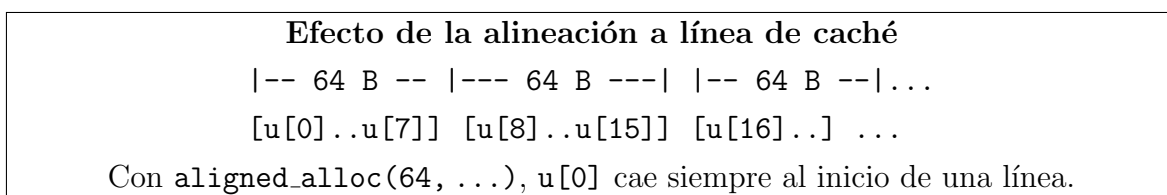


Figura 2: Alineación del arreglo de solución respecto a líneas de caché.

El segundo cambio es el precálculo de h^2 fuera del bucle. Aunque los compiladores modernos pueden realizar esta elevación automáticamente, hacerlo explícitamente garantiza que la multiplicación no se repita en cada iteración independientemente del nivel de optimización.

El núcleo de Jacobi accede a los tres vectores de forma estrictamente secuencial, por lo que el rendimiento está dominado por el ancho de banda de memoria más que por la unidad aritmética. En ese régimen, la alineación tiene mayor impacto que cualquier reducción aritmética.

3.6. Implementación paralela con hilos

La versión con hilos (`threads.c`) divide el dominio entre p hilos POSIX. El estado de la simulación vive en variables globales del proceso (`g_u`, `g_u_new`, `g_f`, `g_h`, `g_n`), visibles directamente por todos los hilos sin mecanismo adicional de memoria compartida.

Partición del dominio. Se usa partición por bloques contiguos: cada hilo t trabaja sobre el rango $[\text{start}_t, \text{end}_t]$ con $\text{chunk} = \lfloor \frac{n}{p} \rfloor$. El último hilo absorbe los elementos sobrantes cuando n no es múltiplo de p . Esta partición mantiene acceso secuencial dentro de cada hilo y es directamente comparable con la variante de procesos.

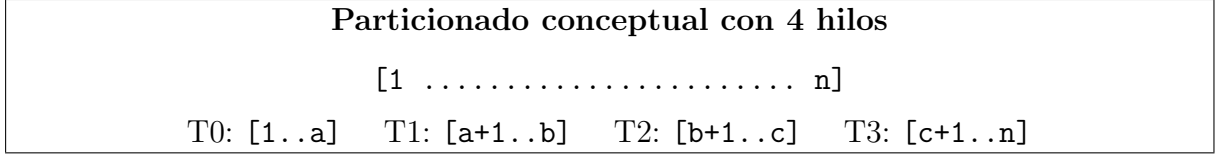


Figura 3: Distribución por bloques contiguos en la versión con hilos.

Sincronización por iteración. Se usan dos barreras `pthread_barrier_wait` en cada iteración:

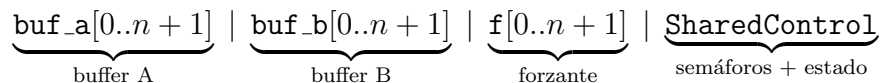
1. Todos los hilos calculan su parte de `u_new`.
2. **Barrera 1:** necesaria porque `rms_residual` accede a `u_new[i-1]` y `u_new[i+1]`, que son escritos por hilos vecinos. Sin esta barrera, un hilo podría leer valores aún no comprometidos por otro.
3. El hilo 0 calcula el residual RMS sobre `g_u_new` completo, actualiza `g_converged` e intercambia los punteros `g_u` y `g_u_new`.
4. **Barrera 2:** garantiza que todos los hilos observan el intercambio de punteros antes de comenzar la siguiente iteración. Sin esta barrera, un hilo podría entrar a la iteración siguiente leyendo el puntero anterior.

Solo el hilo 0 administra convergencia e intercambio de buffers para evitar condiciones de carrera sobre `g_converged` e `g_iters.done`. El costo adicional del modelo aparece en las barreras, que introducen espera cuando hay desbalance de carga o variación de planificación entre hilos.

3.7. Implementación paralela con procesos (fork)

La variante de procesos (`processes.c`) replica el mismo esquema de partición y coordinación que la versión con hilos, pero usando procesos del sistema operativo y memoria compartida explícita. La comparación entre ambas aísla el costo del mecanismo de sincronización y del espacio de direcciones separado, sin diferencias algorítmicas.

Memoria compartida. Como los procesos no comparten espacio de direcciones tras `fork`, todos los datos del solver se alojan en un único bloque reservado con `mmap(MAP_SHARED|MAP_ANONYM)` antes de crear los hijos. El bloque tiene la siguiente disposición:



Usar una sola llamada a `mmap` mantiene los arreglos en páginas físicas contiguas y evita múltiples llamadas al sistema. `SharedControl` contiene los dos semáforos POSIX inicializados con `pshared=1` (modo multiproceso), el flag de intercambio de buffer, el flag de convergencia y el contador de iteraciones.

Arquitectura fork-once. A diferencia de un esquema que crea y destruye procesos en cada iteración, aquí los p trabajadores se crean una sola vez antes del bucle principal mediante un único `fork` en secuencia. Cada hijo ejecuta entonces su propio bucle de iteraciones y se sincroniza con sus pares mediante semáforos dentro de ese bucle. El proceso padre, una vez lanzados los hijos, se limita a esperar su terminación con `waitpid` al final de toda la computación.

Sincronización asimétrica por semáforos. El rol del proceso 0 es equivalente al del hilo 0: actúa como coordinador. El protocolo en cada iteración es el siguiente:

1. Cada proceso calcula su tramo de `u.write` (el buffer determinado por `flip`).
2. Los procesos $1 \dots p-1$ publican en `workers_done` y esperan en `go`.
3. El proceso 0 espera $p-1$ veces en `workers_done`, garantizando que todos han terminado de escribir `u.write` antes de leer valores en los bordes de los tramos al calcular el residual.
4. El proceso 0 calcula el residual RMS sobre `u.write` completo, actualiza `converged` e invierte `flip` con XOR.
5. El proceso 0 publica $p-1$ veces en `go`, permitiendo que los demás avancen a la siguiente iteración.

Este protocolo es estructuralmente equivalente a las dos barreras de la versión con hilos: la espera sobre `workers_done` corresponde a la barrera 1, y la espera sobre `go` corresponde a la barrera 2.

Flag de intercambio de buffer. En la versión con hilos, el intercambio de buffers basta con permutar los punteros globales `g_u` y `g_u_new`, que son visibles por todos los hilos directamente. En la versión con procesos esta estrategia no funciona: los punteros locales de cada proceso son copias independientes tras el `fork`, por lo que modificarlos en un proceso no afecta a los demás. En su lugar, se usa el entero compartido `flip`: en cada iteración cada proceso consulta `flip` para determinar cuál de los dos buffers físicos (`buf_a` o `buf_b`) actúa como lectura y cuál como escritura. El proceso 0 invierte `flip` con una operación XOR al final de cada iteración, y todos los procesos leen el nuevo valor antes de comenzar la siguiente gracias a la barrera implícita de los semáforos.

4. Pruebas

En esta sección se presentan los resultados obtenidos a partir de la ejecución de las tres implementaciones del algoritmo iterativo de Jacobi para la resolución de la ecuación de Poisson 1D: secuencial, concurrente con hilos y paralela con procesos además de las optimizaciones por cache line y por compilador. Además, se incluye un análisis del rendimiento de cada implementación, así como una comparación entre ellas. Para ello, se han utilizado arreglos de diferentes tamaños: 100, 200, 500, 1000 y 2000. Se han realizado 10 ejecuciones para cada tamaño de arreglo y cada implementación, con el fin de obtener resultados más estables y confiables. Se definió un número máximo de iteraciones de 20000000 para asegurar que el algoritmo converge en un punto. Se fijó la tolerancia en 10^{-6} para determinar la convergencia del algoritmo. Se utilizó el mismo entorno de ejecución para todas las pruebas, con el fin de garantizar la comparabilidad de los resultados.

4.1. Resultados del algoritmo secuencial

Se realizaron las pruebas para el algoritmo secuencial y estos fueron los resultados obtenidos:

Secuencial estándar					
Rep	N = 100	N = 200	N = 500	N = 1K	N = 2K
1	13.485	104.554	1,607.344	12,853.034	101,865.591
2	13.473	104.353	1,607.068	12,768.891	101,887.225
3	13.480	108.838	1,606.918	12,760.332	101,883.992
4	13.435	104.659	1,607.093	12,756.565	103,268.970
5	13.457	105.043	1,606.484	12,766.743	101,807.259
6	13.523	108.869	1,606.633	12,860.143	104,178.280
7	13.435	104.370	1,607.711	12,858.129	101,852.757
8	13.449	104.391	1,607.186	12,853.141	101,879.812
9	13.535	104.357	1,608.238	12,757.168	102,672.574
10	13.463	104.299	1,630.843	12,764.037	102,183.326
Promedio	13.474	105.373	1,609.552	12,799.818	102,347.979
Desv. Est.	0.032	1.753	7.113	46.146	758.141
CV (%)	0.24	1.66	0.44	0.36	0.74
Iter. prom.	30,849	122,171	758,964	3,029,711	12,106,565

Cuadro 1: Resultados del algoritmo secuencial

4.2. Resultados del algoritmo secuencial optimizado por cache

Se realizaron las pruebas para el algoritmo secuencial optimizado por cache line y estos fueron los resultados obtenidos:

Secuencial alineado a cache					
Rep	N = 100	N = 200	N = 500	N = 1K	N = 2K
1	13.014	101.029	1,624.725	12,909.829	103,373.955
2	12.968	100.649	1,624.481	12,993.469	103,053.953
3	12.994	107.701	1,624.126	12,899.790	103,318.213
4	12.978	100.694	1,624.149	12,902.386	103,687.709
5	12.995	100.565	1,624.341	13,088.551	103,016.980
6	12.996	100.687	1,628.453	13,014.313	104,689.948
7	12.990	100.652	1,623.857	12,908.891	105,579.004
8	12.996	100.697	1,634.294	12,906.330	103,145.986
9	12.976	100.607	1,623.713	12,894.717	103,154.177
10	12.975	100.647	1,625.170	12,905.244	103,056.091
Promedio	12.988	101.393	1,625.731	12,942.352	103,607.602
Desv. Est.	0.013	2.106	3.134	63.004	811.225
CV (%)	0.10	2.08	0.19	0.49	0.78
Iter. prom.	30,849	122,171	758,964	3,029,711	12,106,565

Cuadro 2: Resultados del algoritmo secuencial optimizado por cache line

Tabla 2 — Promedio y Speedup (ref: serial_std)											
Implementación	N=100 Avg (ms)	N=100 Speedup	N=200 Avg (ms)	N=200 Speedup	N=500 Avg (ms)	N=500 Speedup	N=1K Avg (ms)	N=1K Speedup	N=2K Avg (ms)	N=2K Speedup	Speedup Prom.
Secuencial estándar	13.474	1.0000	105.373	1.0000	1,609.552	1.0000	12,799.818	1.0000	102,347.979	1.0000	1.00
Secuencial optimizado (-O3 full)	1.035	13.0191	7.245	14.5437	136.125	11.8240	967.214	13.2337	8,819.669	11.6045	12.85
Secuencial alineado a cache	12.988	1.0374	101.393	1.0393	1,625.731	0.9900	12,942.352	0.9890	103,607.602	0.9878	1.01

Cuadro 3: Speedup del algoritmo secuencial optimizado por cache line

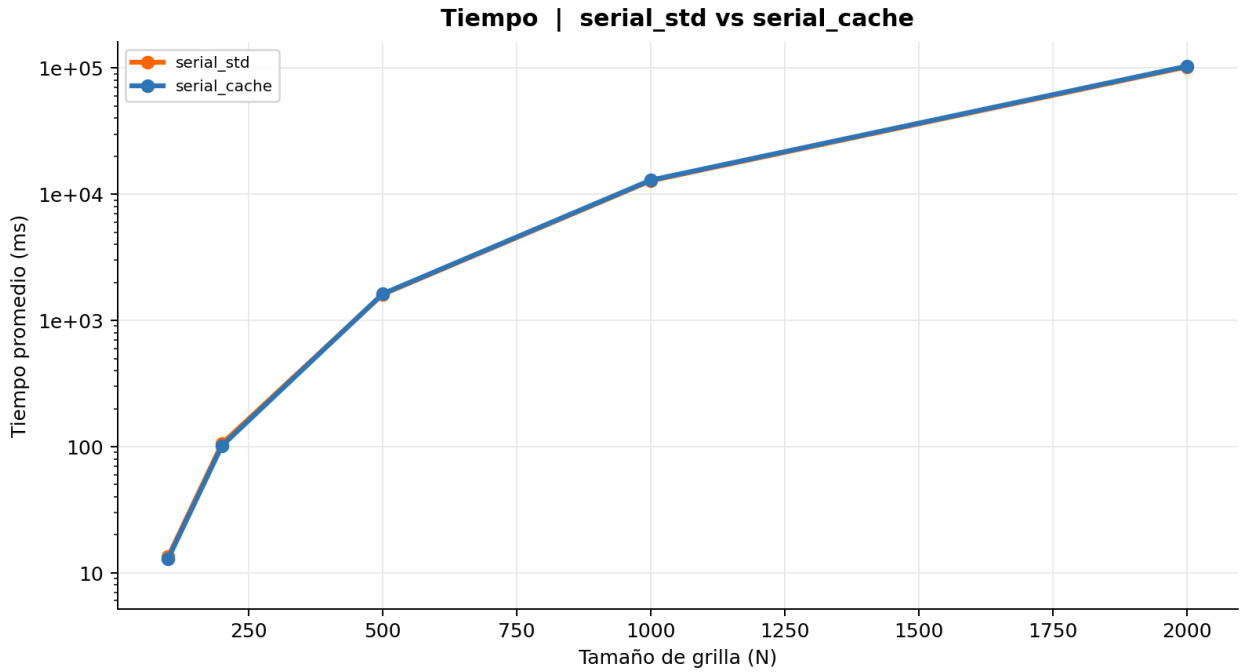


Figura 4: Comparación del tiempo de ejecución entre el algoritmo secuencial y el algoritmo secuencial optimizado por cache line

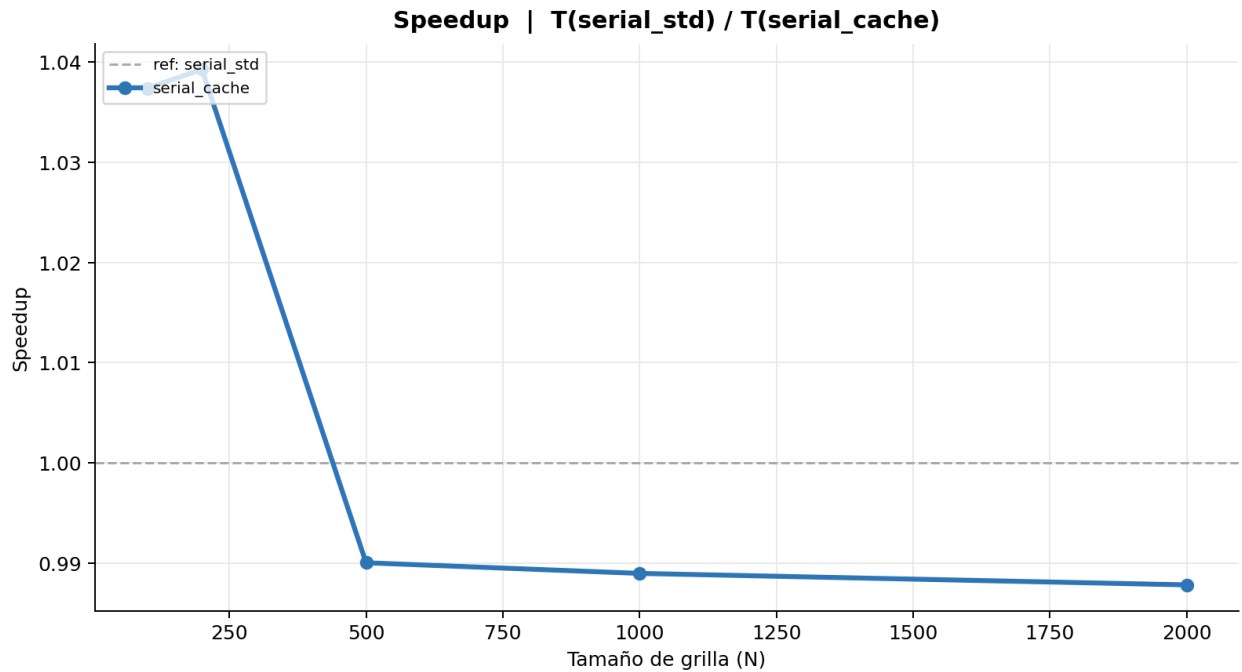


Figura 5: Comparación del speedup entre el algoritmo secuencial y el algoritmo secuencial optimizado por cache line

4.3. Resultados de las pruebas del algoritmo optimizado por compilador

Para este algoritmo se usaron las siguientes opciones de compilación: `-O3`, `-funroll-loops`, `-fomit-frame-pointer` y `-march=native`. Estas opciones permiten al compilador realizar optimizaciones agresivas, como la eliminación de código innecesario, el desenrollado de bucles y la generación de código específico para la arquitectura del procesador. Estas optimizaciones pueden mejorar significativamente el rendimiento del algoritmo, especialmente en casos donde se realizan muchas iteraciones o se manejan grandes cantidades de datos.

El significado de cada bandera es el siguiente:

- **-O3:** Esta bandera habilita un nivel de optimización agresivo, permitiendo al compilador aplicar una amplia gama de técnicas para mejorar el rendimiento del código.
- **-Wall:** Activa la mayoría de las advertencias del compilador, lo que ayuda a identificar posibles problemas en el código que podrían afectar su rendimiento o funcionalidad.
- **-march=native:** Permite al compilador generar código optimizado para la arquitectura específica del procesador en el que se está compilando, aprovechando las características y capacidades de hardware disponibles.
- **-funroll-loops:** Habilita el desenrollado de bucles, lo que puede mejorar el rendimiento al reducir la sobrecarga de control de bucle y aumentar la paralelización de las instrucciones.
- **-flto:** Habilita la optimización a nivel de enlace, lo que permite al compilador realizar optimizaciones globales en todo el programa, incluso entre diferentes archivos de código fuente.
- **-ffast-math:** Permite al compilador realizar optimizaciones agresivas en operaciones matemáticas, lo que puede mejorar el rendimiento a costa de la precisión en algunos casos como la evaluación de funciones trigonométricas.
- **-fomit-frame-pointer:** Indica al compilador que no utilice un puntero de marco para las funciones, es decir, que no cree un marco de pila para cada función, lo que puede mejorar el rendimiento al reducir la cantidad de instrucciones necesarias para gestionar la pila de llamadas.

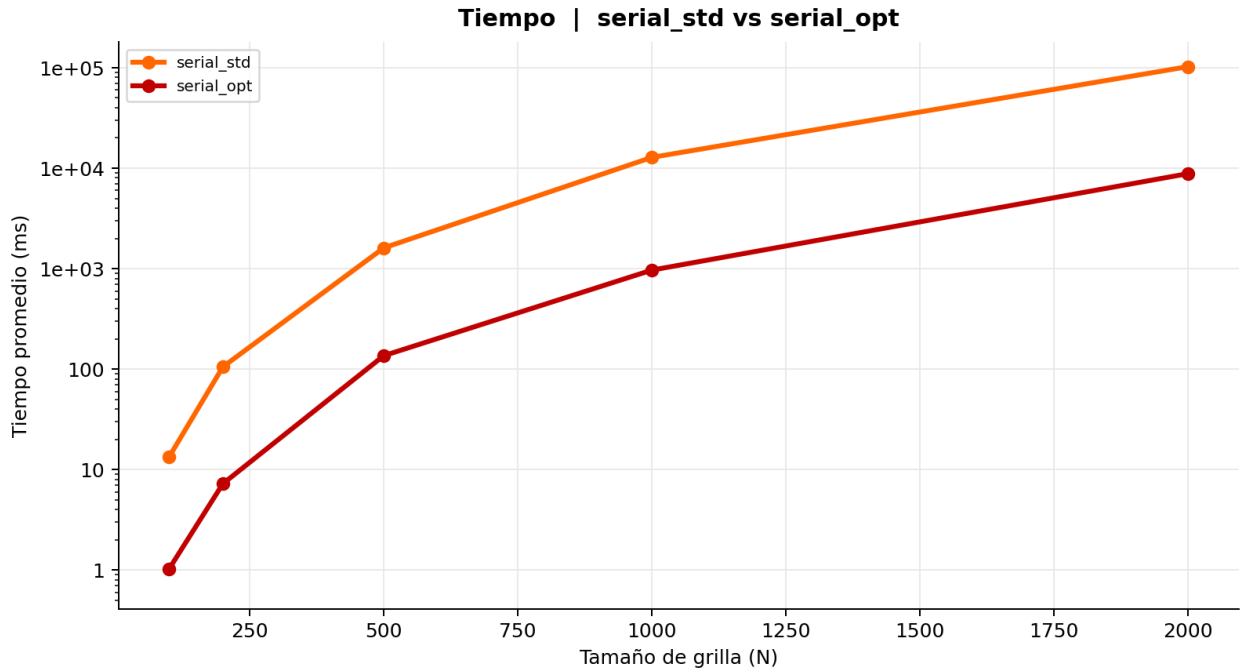
Se realizaron las pruebas para el algoritmo secuencia optimizado por compilador y estos fueron los resultados obtenidos:

Secuencial optimizado (-O3 full)					
Rep	N = 100	N = 200	N = 500	N = 1K	N = 2K
1	1.012	7.394	115.563	921.707	8,757.090
2	1.085	7.217	116.986	1,148.103	8,768.079
3	1.101	7.265	116.326	922.216	8,775.448
4	1.025	7.302	116.660	921.968	8,762.380
5	1.070	7.169	116.468	1,147.471	9,021.799
6	1.007	7.353	116.289	921.831	8,848.660
7	0.990	7.176	116.820	921.562	8,949.440
8	1.015	7.195	116.716	922.439	8,798.499
9	1.013	7.119	116.733	921.480	8,757.029
10	1.031	7.263	312.693	923.362	8,758.265
Promedio	1.035	7.245	136.125	967.214	8,819.669
Desv. Est.	0.035	0.082	58.857	90.288	88.597
CV (%)	3.41	1.13	43.24	9.33	1.00
Iter. prom.	30,849	122,171	758,964	3,029,711	12,106,565

Cuadro 4: Resultados del algoritmo secuencial optimizado por compilador

Tabla 2 — Promedio y Speedup (ref: serial_std)											
Implementación	N=100 Avg (ms)	N=100 Speedup	N=200 Avg (ms)	N=200 Speedup	N=500 Avg (ms)	N=500 Speedup	N=1K Avg (ms)	N=1K Speedup	N=2K Avg (ms)	N=2K Speedup	Speedup Prom.
Secuencial estándar	13.474	1.0000	105.373	1.0000	1,609.552	1.0000	12,799.818	1.0000	102,347.979	1.0000	1.00
Secuencial optimizado (-O3 full)	1.035	13.0191	7.245	14.5437	136.125	11.8240	967.214	13.2337	8,819.669	11.6045	12.85
Secuencial alineado a cache	12.988	1.0374	101.393	1.0393	1,625.731	0.9900	12,942.352	0.9890	103,607.602	0.9878	1.01

Cuadro 5: Speedup del algoritmo secuencial optimizado por compilador



Cuadro 6: Comparación del tiempo de ejecución entre el algoritmo secuencial y el algoritmo secuencial optimizado por compilador

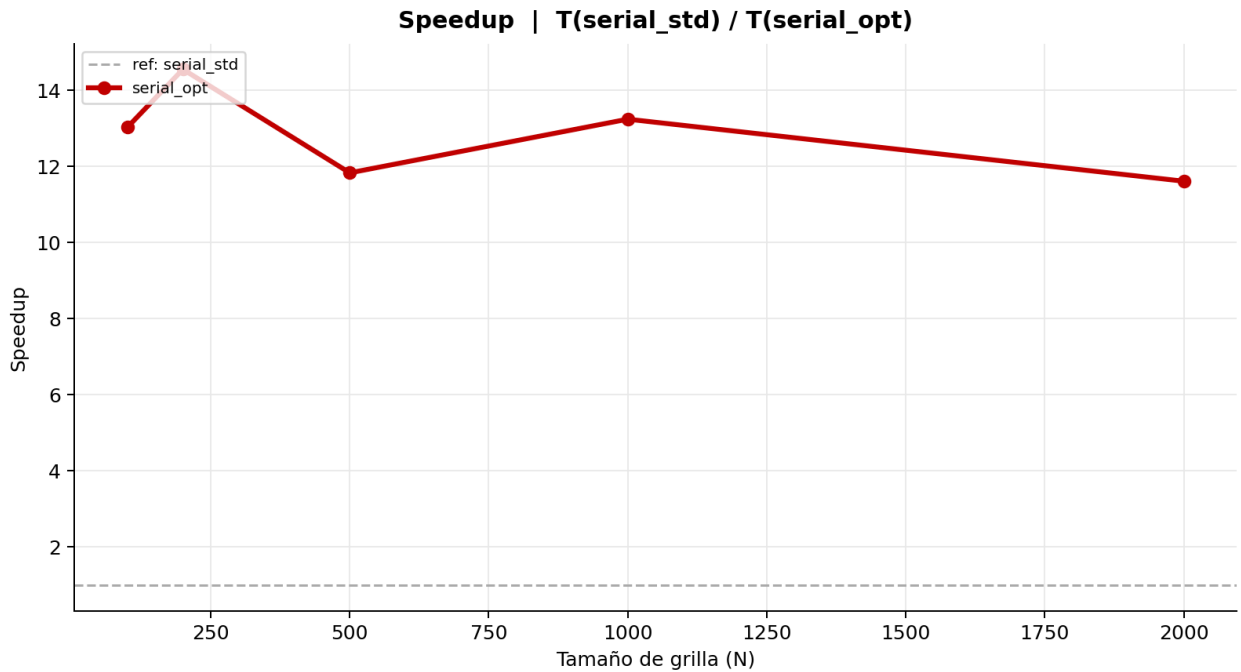


Figura 6: Comparación del speedup entre el algoritmo secuencial y el algoritmo secuencial optimizado por compilador

4.4. Resultados del algoritmo paralelizado con hilos

Para esta implementación se utilizó la biblioteca de hilos POSIX (pthreads) para crear múltiples hilos que ejecutan el algoritmo de Jacobi en paralelo. Cada hilo se encarga de procesar una parte del arreglo, lo que permite aprovechar la capacidad de procesamiento de múltiples núcleos del CPU. Se realizaron pruebas con diferentes números de hilos (2, 4, 6, 8 y 12) para evaluar el impacto en el rendimiento. Estos fueron los resultados obtenidos:

Hilos Jacobi (2 hilos)					
Rep	N = 100	N = 200	N = 500	N = 1K	N = 2K
1	157.822	705.245	5,372.185	27,011.860	153,210.044
2	160.471	710.206	5,447.951	27,212.804	152,927.110
3	157.649	703.300	5,343.549	27,063.949	153,908.703
4	156.590	699.754	5,417.084	27,158.230	153,717.194
5	158.136	698.422	5,347.245	27,091.218	152,709.856
6	157.820	703.807	5,340.632	27,065.733	152,954.619
7	155.843	706.846	5,352.716	27,092.917	152,498.490
8	158.289	693.350	5,323.858	27,074.253	153,681.982
9	156.311	701.442	5,339.348	26,956.859	153,184.315
10	155.688	697.576	5,334.491	27,052.981	152,823.486
Promedio	157.462	701.995	5,361.906	27,078.080	153,161.580
Desv. Est.	1.354	4.678	37.883	67.115	446.816
CV (%)	0.86	0.67	0.71	0.25	0.29
Iter. prom.	30,849	122,171	758,964	3,029,711	12,106,565

Cuadro 7: Resultados del algoritmo paralelizado con hilos

Hilos Jacobi (4 hilos)					
Rep	N = 100	N = 200	N = 500	N = 1K	N = 2K
1	176.127	730.492	5,436.843	26,982.576	147,931.982
2	177.852	732.197	5,457.987	27,063.888	147,160.055
3	175.164	730.108	5,486.308	27,034.641	147,862.817
4	176.077	729.006	5,429.368	26,953.470	147,398.610
5	176.904	732.678	5,470.165	26,977.122	147,605.403
6	176.729	727.326	5,398.750	26,951.863	147,145.051
7	175.893	722.088	5,414.286	26,972.009	147,709.300
8	176.830	723.678	5,397.974	26,980.774	147,580.360
9	176.140	730.826	5,404.481	26,898.525	147,732.255
10	176.243	726.244	5,408.015	26,963.269	145,472.275
Promedio	176.396	728.464	5,430.418	26,977.814	147,359.811
Desv. Est.	0.686	3.379	29.994	42.974	678.272
CV (%)	0.39	0.46	0.55	0.16	0.46
Iter. prom.	30,849	122,171	758,964	3,029,711	12,106,565

Cuadro 8: Resultados del algoritmo paralelizado con hilos

Hilos Jacobi (6 hilos)					
Rep	N = 100	N = 200	N = 500	N = 1K	N = 2K
1	186.486	768.488	5,657.680	26,468.021	140,077.056
2	185.224	765.488	5,620.728	26,540.348	140,503.313
3	186.842	770.199	5,776.196	26,421.040	142,347.985
4	185.101	766.516	5,630.878	26,191.487	141,003.304
5	186.026	769.096	5,622.571	26,757.165	142,963.499
6	185.464	766.343	5,607.573	26,566.649	140,281.447
7	184.770	763.260	5,616.388	26,427.430	140,647.017
8	184.682	763.273	5,616.987	25,642.611	141,145.810
9	183.745	764.800	5,600.273	26,090.978	141,611.742
10	184.129	761.340	5,594.978	25,909.752	139,877.442
Promedio	185.247	765.880	5,634.425	26,301.548	141,045.861
Desv. Est.	0.938	2.683	50.053	322.007	951.127
CV (%)	0.51	0.35	0.89	1.22	0.67
Iter. prom.	30,849	122,171	758,964	3,029,711	12,106,565

Cuadro 9: Resultados del algoritmo paralelizado con hilos

Hilos Jacobi (8 hilos)					
Rep	N = 100	N = 200	N = 500	N = 1K	N = 2K
1	372.701	1,490.956	9,751.340	41,920.319	180,635.434
2	366.051	1,474.892	9,802.560	42,103.542	181,313.474
3	368.484	1,465.042	9,795.067	42,080.392	180,736.287
4	369.055	1,471.761	9,794.885	42,032.315	181,154.813
5	370.719	1,494.518	9,703.643	41,977.259	180,891.438
6	374.330	1,483.212	9,814.812	42,260.191	181,526.651
7	366.340	1,471.925	9,776.524	42,201.921	181,477.879
8	367.717	1,482.406	9,805.043	41,988.741	180,668.672
9	370.324	1,494.635	9,756.911	42,051.566	181,625.174
10	369.764	1,489.910	9,852.763	42,073.742	181,678.829
Promedio	369.548	1,481.926	9,785.355	42,068.999	181,170.865
Desv. Est.	2.492	10.026	38.731	97.047	388.804
CV (%)	0.67	0.68	0.40	0.23	0.21
Iter. prom.	30,849	122,171	758,964	3,029,711	12,106,565

Cuadro 10: Resultados del algoritmo paralelizado con hilos

Hilos Jacobi (12 hilos)					
Rep	N = 100	N = 200	N = 500	N = 1K	N = 2K
1	537.436	2,120.820	13,490.437	56,718.615	262,992.086
2	528.062	2,111.218	13,452.715	56,745.200	263,234.116
3	527.627	2,110.245	13,441.654	56,857.548	262,705.520
4	531.344	2,110.730	13,451.734	56,750.654	263,345.750
5	529.955	2,105.150	13,456.565	56,873.242	263,393.686
6	526.976	2,109.049	13,449.475	56,771.259	263,191.397
7	529.474	2,108.304	13,442.232	56,775.843	263,638.572
8	528.715	2,107.598	13,428.584	56,921.664	263,179.819
9	527.347	2,106.016	13,402.554	56,635.342	263,084.218
10	526.409	2,101.832	13,390.045	56,697.636	262,657.147
Promedio	529.335	2,109.096	13,440.600	56,774.700	263,142.231
Desv. Est.	3.051	4.761	26.906	82.547	285.519
CV (%)	0.58	0.23	0.20	0.15	0.11
Iter. prom.	30,849	122,171	758,964	3,029,711	12,106,565

Cuadro 11: Resultados del algoritmo paralelizado con hilos

Implementación	N=100 Avg (ms)	N=100 Speedup	N=200 Avg (ms)	N=200 Speedup	N=500 Avg (ms)	N=500 Speedup	N=1K Avg (ms)	N=1K Speedup	N=2K Avg (ms)	N=2K Speedup	Speedup Prom.
Secuencial estándar	13.474	1.0000	105.373	1.0000	1,609.552	1.0000	12,799.818	1.0000	102,347.979	1.0000	1.00
Hilos Jacobi (2 hilos)	157.462	0.0856	701.995	0.1501	5,361.906	0.3002	27,078.080	0.4727	153,161.580	0.6682	0.34
Hilos Jacobi (4 hilos)	176.396	0.0764	728.464	0.1447	5,430.418	0.2964	26,977.814	0.4745	147,359.811	0.6945	0.34
Hilos Jacobi (6 hilos)	185.247	0.0727	765.880	0.1376	5,634.425	0.2857	26,301.548	0.4867	141,045.861	0.7256	0.34
Hilos Jacobi (8 hilos)	369.548	0.0365	1,481.926	0.0711	9,785.355	0.1645	42,068.999	0.3043	181,170.865	0.5649	0.23
Hilos Jacobi (12 hilos)	529.335	0.0255	2,109.096	0.0500	13,440.600	0.1198	56,774.700	0.2254	263,142.231	0.3889	0.16

Cuadro 12: Speedup del algoritmo paralelizado con hilos

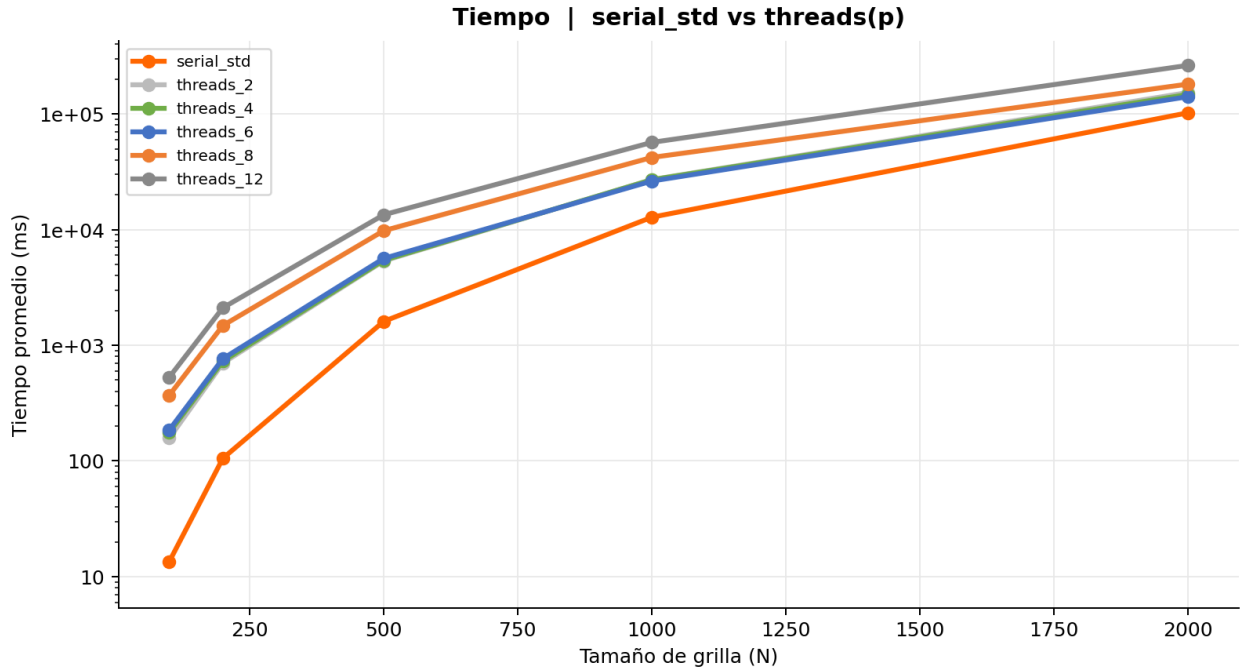


Figura 7: Comparación del tiempo de ejecución entre el algoritmo secuencial y el algoritmo paralelizado con hilos

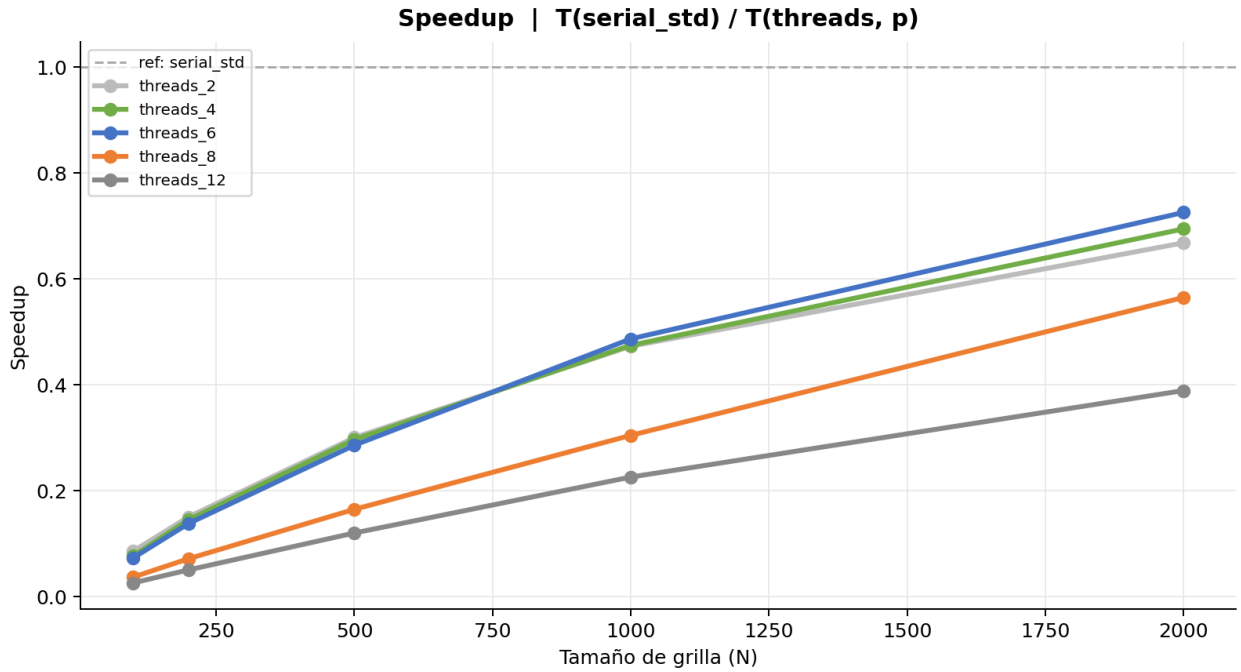


Figura 8: Comparación del speedup entre el algoritmo secuencial y el algoritmo paralelizado con hilos

4.5. Resultados del algoritmo paralelizado por procesos

Para esta implementación se utilizó la biblioteca de procesos POSIX (fork) para crear múltiples procesos que ejecutan el algoritmo de Jacobi en paralelo. Cada proceso se encarga de procesar una parte del arreglo, lo que permite aprovechar la capacidad de procesamiento de múltiples núcleos del CPU. Se realizaron pruebas con diferentes números de procesos (2, 4, 6, 8 y 12) alienados a las pruebas con hilos realizadas para evaluar el impacto en el rendimiento. Estos fueron los resultados obtenidos:

Procesos fork (2 procesos)					
Rep	N = 100	N = 200	N = 500	N = 1K	N = 2K
1	159.314	647.072	5,061.576	24,060.141	134,730.831
2	151.236	643.892	4,889.828	24,161.518	134,779.447
3	151.554	649.976	4,952.508	24,077.896	134,084.360
4	150.497	621.791	4,846.296	24,009.528	139,853.408
5	153.805	620.181	4,920.931	24,299.055	135,015.511
6	159.260	615.440	4,912.541	24,322.951	135,984.363
7	159.726	643.312	4,927.486	24,242.024	135,430.301
8	150.838	657.887	4,896.349	24,162.075	135,367.365
9	153.953	621.855	4,909.384	24,276.520	135,957.822
10	155.284	632.432	4,906.857	24,284.873	154,912.622
Promedio	154.547	635.384	4,922.376	24,189.658	137,611.603
Desv. Est.	3.511	14.130	53.239	105.997	5,958.436
CV (%)	2.27	2.22	1.08	0.44	4.33
Iter. prom.	30,849	122,171	758,964	3,029,711	12,106,565

Cuadro 13: Resultados del algoritmo paralelizado por procesos

Procesos fork (4 procesos)					
Rep	N = 100	N = 200	N = 500	N = 1K	N = 2K
1	290.129	839.794	22,056.950	30,111.016	137,805.737
2	321.441	993.066	21,799.655	27,029.179	138,231.354
3	275.659	953.205	22,784.581	25,341.042	137,798.848
4	283.415	987.107	21,663.789	25,738.205	138,295.950
5	285.810	909.934	22,015.519	24,770.663	138,199.389
6	296.703	1,018.015	24,033.877	25,087.388	137,705.741
7	310.821	984.412	21,715.531	25,049.319	137,865.867
8	276.985	904.425	21,882.521	24,952.263	138,062.677
9	302.348	1,015.917	21,937.108	24,981.268	137,890.385
10	287.975	908.090	19,830.721	24,805.844	139,015.993
Promedio	293.129	951.397	21,972.025	25,786.619	138,087.194
Desv. Est.	14.006	55.643	985.165	1,575.329	365.487
CV (%)	4.78	5.85	4.48	6.11	0.26
Iter. prom.	57,136	171,041	3,377,076	3,174,419	12,148,307

Cuadro 14: Resultados del algoritmo paralelizado por procesos

Procesos fork (6 procesos)					
Rep	N = 100	N = 200	N = 500	N = 1K	N = 2K
1	294.936	1,195.328	11,220.918	66,772.918	—
2	283.815	1,161.403	10,088.374	57,658.142	—
3	254.485	1,187.386	10,578.748	51,839.525	—
4	261.036	1,016.967	10,405.703	47,730.736	—
5	248.115	1,028.491	10,470.363	51,954.916	—
6	256.771	1,100.745	11,114.013	53,032.826	—
7	269.905	1,141.019	10,571.046	52,808.613	—
8	247.720	1,097.421	10,742.955	52,088.145	—
9	245.667	970.279	10,542.420	49,781.896	—
10	264.814	987.454	10,293.134	50,434.820	—
Promedio	262.726	1,088.649	10,602.767	53,410.254	—
Desv. Est.	15.408	78.992	329.424	5,076.594	—
CV (%)	5.86	7.26	3.11	9.50	—
Iter. prom.	70,733	300,590	2,573,935	6,751,089	—

Cuadro 15: Resultados del algoritmo paralelizado por procesos

Procesos fork (8 procesos)					
Rep	N = 100	N = 200	N = 500	N = 1K	N = 2K
1	425.559	1,998.938	15,910.887	86,719.504	—
2	465.797	1,862.220	15,491.049	73,890.344	—
3	406.269	2,217.005	17,011.323	89,812.246	—
4	451.819	2,224.230	15,935.354	96,359.060	—
5	490.776	2,218.694	16,694.387	97,835.363	—
6	509.150	2,176.210	18,328.084	97,220.106	—
7	550.375	2,070.998	17,922.875	116,670.63	—
8	538.732	2,283.960	18,710.304	99,477.422	—
9	501.710	2,335.755	18,544.987	93,597.995	—
10	562.484	2,044.420	16,494.355	116,166.65	—
Promedio	490.267	2,143.243	17,104.361	96,774.934	—
Desv. Est.	49.985	137.994	1,129.801	12,099.856	—
CV (%)	10.20	6.44	6.61	12.50	—
Iter. prom.	71,747	310,660	2,317,058	11,664,547	—

Cuadro 16: Resultados del algoritmo paralelizado por procesos

Procesos fork (12 procesos)					
Rep	N = 100	N = 200	N = 500	N = 1K	N = 2K
1	795.912	3,070.911	23,955.298	114,413.125	—
2	724.090	3,185.604	22,581.335	112,752.926	—
3	777.981	3,280.217	24,358.039	123,514.787	—
4	777.574	2,892.879	21,933.515	143,253.147	—
5	774.697	2,968.101	22,369.455	124,252.036	—
6	756.897	3,202.824	24,033.051	112,101.691	—
7	765.816	3,361.423	23,028.248	125,928.150	—
8	812.193	2,994.351	22,556.966	108,919.229	—
9	782.176	3,157.736	27,064.323	109,451.341	—
10	741.160	2,957.610	24,436.470	107,050.923	—
Promedio	770.850	3,107.166	23,631.670	118,163.735	—
Desv. Est.	24.217	146.328	1,425.605	10,584.987	—
CV (%)	3.14	4.71	6.03	8.96	—
Iter. prom.	76,108	303,976	2,185,791	9,777,002	—

Cuadro 17: Resultados del algoritmo paralelizado por procesos

Tabla 2 — Promedio y Speedup (ref: serial_std)											
Implementación	N=100 Avg (ms)	N=100 Speedup	N=200 Avg (ms)	N=200 Speedup	N=500 Avg (ms)	N=500 Speedup	N=1K Avg (ms)	N=1K Speedup	N=2K Avg (ms)	N=2K Speedup	Speedup Prom.
Secuencial estándar	13.474	1.0000	105.373	1.0000	1,609.552	1.0000	12,799.818	1.0000	102,347.979	1.0000	1.00
Procesos fork (2 procesos)	154.547	0.0872	635.384	0.1658	4,922.376	0.3270	24,189.658	0.5291	137,611.603	0.7437	0.37
Procesos fork (4 procesos)	293.129	0.0460	951.397	0.1108	21,972.025	0.0733	25,786.619	0.4964	138,087.194	0.7412	0.29
Procesos fork (6 procesos)	262.726	0.0513	1,088.649	0.0968	10,602.767	0.1518	53,410.254	0.2397	N/A	N/A	0.13
Procesos fork (8 procesos)	490.267	0.0275	2,143.243	0.0492	17,104.361	0.0941	96,774.934	0.1323	N/A	N/A	0.08
Procesos fork (12 procesos)	770.850	0.0175	3,107.166	0.0339	23,631.670	0.0681	118,163.735	0.1083	N/A	N/A	0.06

Cuadro 18: Speedup del algoritmo paralelizado por procesos

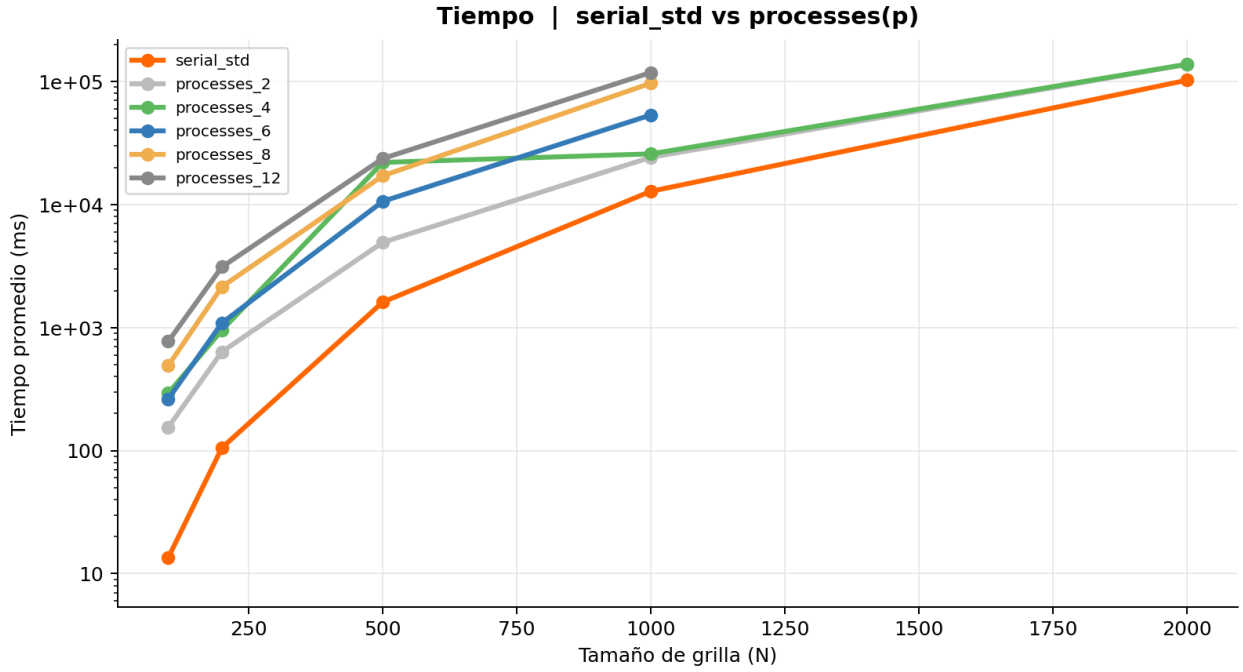


Figura 9: Comparación del tiempo de ejecución entre el algoritmo secuencial y el algoritmo paralelizado por procesos

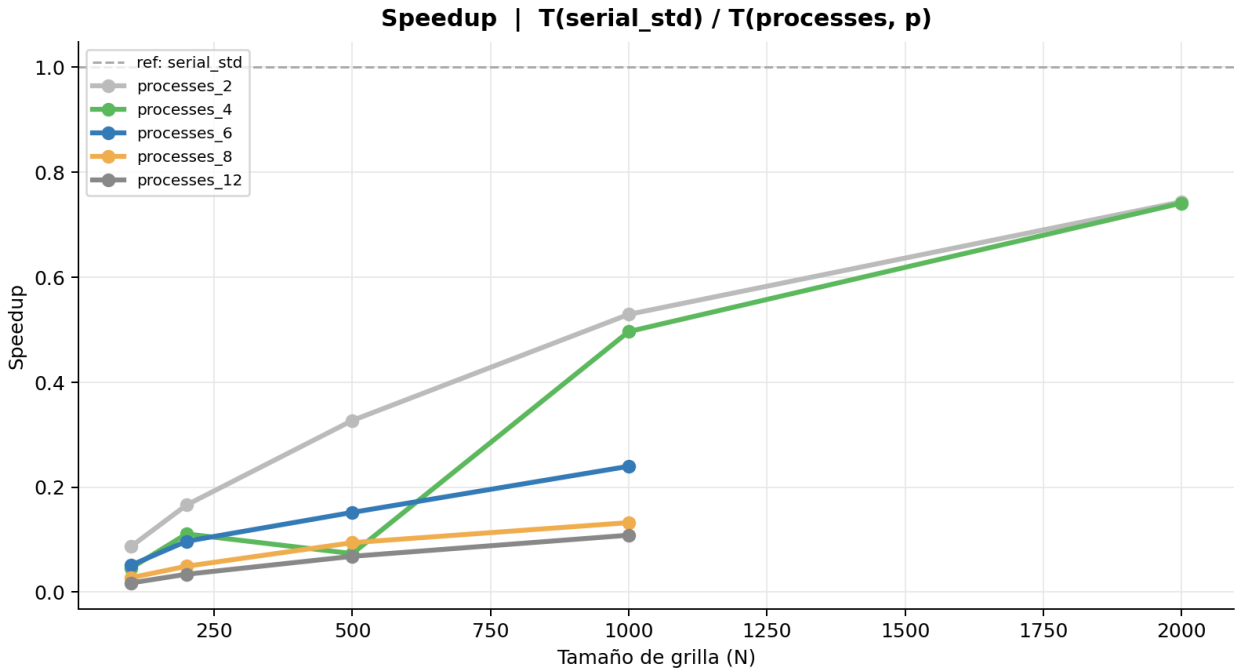


Figura 10: Comparación del speedup entre el algoritmo secuencial y el algoritmo paralelizado por procesos

Para esta prueba se puede observar que para $p \geq 6$ no fue posible obtener resultados para

$n = 2000$ en un tiempo razonable.

Cuando se ejecutan 6 procesos en una máquina de 6 núcleos, todos los núcleos quedan completamente ocupados. El problema es que en cada iteración del algoritmo, el proceso coordinador necesita despertar a los demás procesos mediante semáforos. Despertar un proceso en el sistema operativo tiene un costo, y ese costo se multiplica cuando no hay núcleos disponibles: el sistema operativo debe interrumpir forzosamente a uno de los procesos en ejecución para cederle el turno al proceso recién despertado. Esta operación ocurre 5 veces por iteración, y para $n = 2000$ el algoritmo requiere aproximadamente 12 millones de iteraciones para converger, lo que hace que el overhead acumulado de sincronización supere la hora de ejecución por repetición.

La implementación con hilos no presenta este problema porque los hilos pertenecen al mismo proceso y el sistema operativo los gestiona de forma más ligera, sin necesidad de interrumpir otros núcleos para sincronizarlos. Esto se confirma en los datos: con $p = 6$ y $n = 2000$, la versión con hilos completó en aproximadamente 141 segundos, mientras que la versión con procesos no logró terminar en más de una hora de ejecución.

Por esta razón, los resultados de la suite de procesos para $p \geq 6$ se reportan únicamente hasta $n = 1000$.

5. Comparaciones

En esta sección se presentan comparaciones entre los diferentes algoritmos implementados, incluyendo el algoritmo secuencial optimizado por compilador, el algoritmo secuencial optimizado por cache line, el algoritmo paralelizado con hilos y el algoritmo paralelizado por procesos. Se analizarán los resultados obtenidos en términos de tiempo de ejecución y speedup para cada caso, destacando las diferencias y similitudes entre las distintas implementaciones. Además, se discutirán las posibles razones detrás de los resultados observados y se ofrecerán conclusiones sobre el rendimiento de cada enfoque en función de los datos recopilados durante las pruebas.

Tabla 2 — Promedio y Speedup (ref: serial_std)											
Implementación	N=100 Avg (ms)	N=100 Speedup	N=200 Avg (ms)	N=200 Speedup	N=500 Avg (ms)	N=500 Speedup	N=1K Avg (ms)	N=1K Speedup	N=2K Avg (ms)	N=2K Speedup	Speedup Prom.
Secuencial estándar	13.474	1.0000	105.373	1.0000	1,609.552	1.0000	12,799.818	1.0000	102,347.979	1.0000	1.00
Secuencial optimizado (-O3 full)	1.035	13.0191	7.245	14.5437	136.125	11.8240	967.214	13.2337	8,819.669	11.6045	12.85
Secuencial alineado a cache	12.988	1.0374	101.393	1.0393	1,625.731	0.9900	12,942.352	0.9890	103,607.602	0.9878	1.01
Hilos Jacobi (6 hilos)	185.247	0.0727	765.880	0.1376	5,634.425	0.2857	26,301.548	0.4867	141,045.861	0.7256	0.34
Procesos fork (2 procesos)	154.547	0.0872	635.384	0.1658	4,922.376	0.3270	24,189.658	0.5291	137,611.603	0.7437	0.37

Cuadro 19: Comparación del speedup entre los diferentes algoritmos

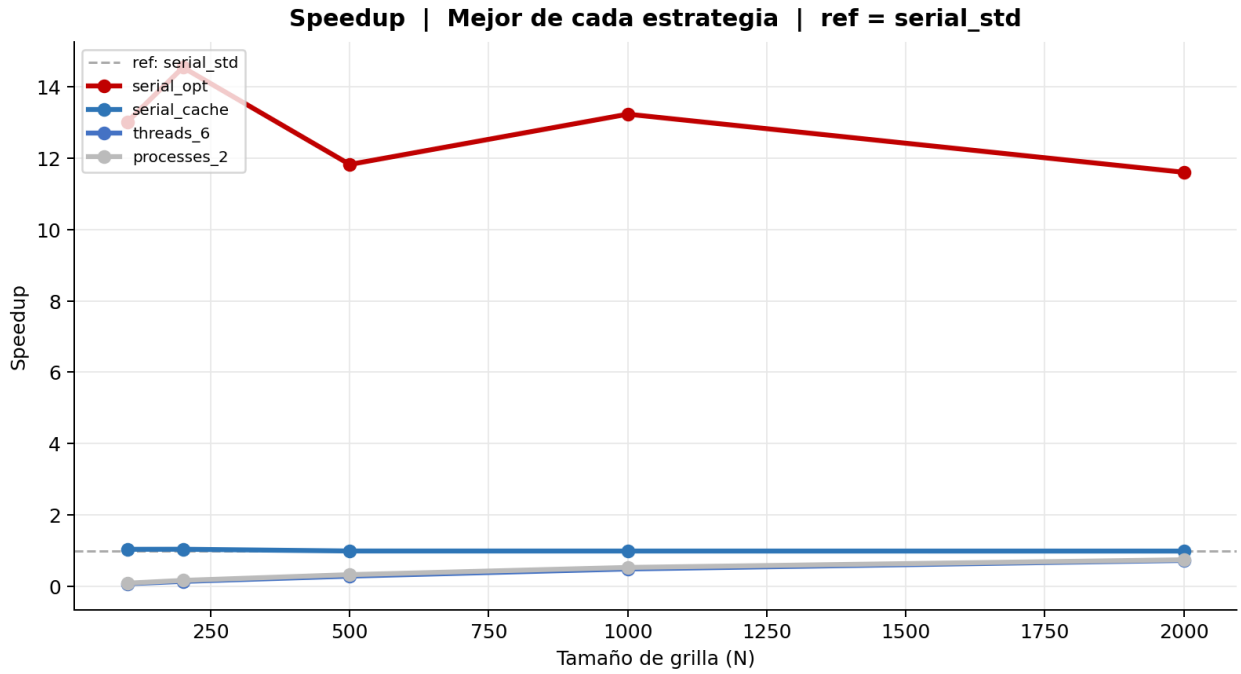


Figura 11: Comparación del speedup entre los diferentes algoritmos

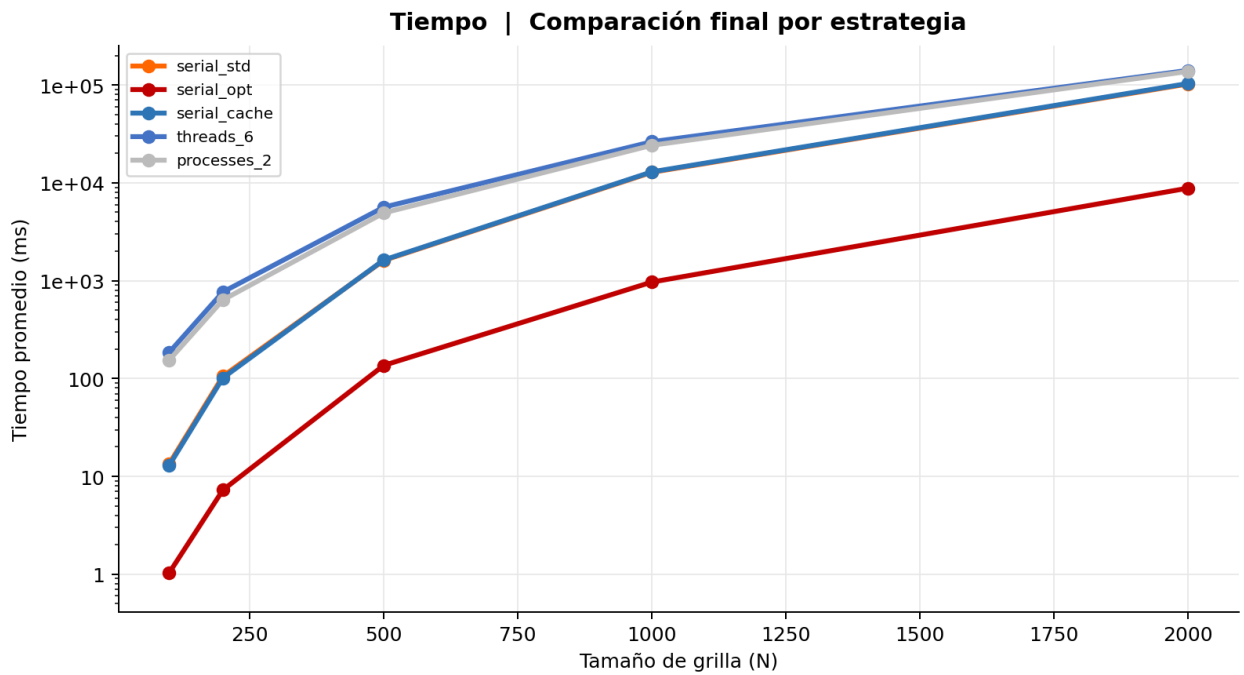


Figura 12: Comparación del tiempo de ejecución entre los diferentes algoritmos

6. Conclusiones

1. La implementación secuencial optimizada por compilador alcanza entre $11,6\times$ y $14,5\times$ de speedup sobre la implementación secuencial base, mientras que ninguna configuración paralela supera $0,74\times$. Esto se debe a que las optimizaciones de compilador activan la vectorización AVX2, que permite procesar cuatro valores en paralelo dentro del mismo núcleo, sin ningún costo de sincronización. Antes de recurrir a la paralelización, vale la pena asegurarse de que el código serial esté bien optimizado.
2. El algoritmo de Jacobi 1D no se beneficia de la paralelización para los tamaños de problema evaluados: ninguna implementación paralela supera a la versión secuencial base. La razón es que el cálculo del residuo se ejecuta en un solo hilo mientras los demás esperan, y ese trabajo tiene el mismo costo que el que realiza cada hilo en su porción del arreglo, $\mathcal{O}(n/p)$. Según la Ley de Amdahl, esta fracción serial limita el speedup máximo alcanzable sin importar cuántos hilos se agreguen.
3. A medida que n crece, el cómputo útil por iteración empieza a compensar el costo fijo de sincronización. Para la implementación con hilos y $p = 6$, el speedup pasa de $0,073\times$ en $n = 100$ a $0,726\times$ en $n = 2000$. Aun así, con los tamaños evaluados nunca se supera la línea de $1\times$.
4. La configuración óptima para la implementación con hilos es $p = 6$, lo que coincide exactamente con los seis núcleos físicos del procesador utilizado. Al pasar a $p = 8$, el speedup cae: para $n = 1000$, baja de $0,487\times$ a $0,304\times$. Los hilos adicionales comparten el mismo núcleo físico y compiten por los mismos recursos de ejecución, por lo que el Hyperthreading no aporta ningún beneficio en este caso.
5. Para $p = 2$, hilos y procesos producen tiempos muy similares en todos los valores de n . A partir de $p = 4$, los procesos empiezan a ser considerablemente más lentos, y la brecha se amplía con p . El caso más notable se da en $n = 1000$ con $p = 12$: la implementación con hilos tarda 56,775 ms frente a 118,164 ms de la implementación con procesos, es decir, $2,1\times$ más lento.
6. La implementación secuencial con alineación explícita de memoria a 64 bytes no muestra ninguna mejora significativa respecto a la versión base, con speedups entre $0,988\times$ y $1,039\times$. Para los tamaños de problema evaluados, los arreglos caben en los niveles L2 y L3 de caché, y el patrón de acceso secuencial es suficiente para que el procesador gestione los datos de forma eficiente por sí solo.

Referencias

- Barrett, R., Berry, M., Chan, T. F., Demmel, J., Donato, J., Dongarra, J., Eijkhout, V., Pozo, R., Romine, C., & van der Vorst, H. (1994). *Templates for the Solution of Linear Systems: Building Blocks for Iterative Methods*. SIAM. <https://doi.org/10.1137/1.9781611971538>
- Ben-Ari, M. (2006). *Principles of Concurrent and Distributed Programming* (2nd). Addison-Wesley.
- Briggs, W. L., Henson, V. E., & McCormick, S. F. (2000). *A Multigrid Tutorial* (2nd). SIAM. <https://doi.org/10.1137/1.9780898719505>
- Burkardt, J. (2011). *Jacobi Iterative Solution of Poisson's Equation in 1D* (inf. téc.) (Disponible en: http://people.sc.fsu.edu/~jburkardt/presentations/jacobi_poisson_1d.pdf). Department of Scientific Computing, Florida State University.
- Drepper, U. (2007). What Every Programmer Should Know About Memory [Disponible en: <https://www.akkadia.org/drepper/cpumemory.pdf>].
- Elman, H. C., Silvester, D. J., & Wathen, A. J. (2005). *Finite Elements and Fast Iterative Solvers with Applications in Incompressible Fluid Dynamics*. Oxford University Press.
- Gustafson, J. L. (1988). Reevaluating Amdahl's Law. *Communications of the ACM*, 31(5), 532-533. <https://doi.org/10.1145/42411.42415>
- Hager, G., & Wellein, G. (2010). *Introduction to High Performance Computing for Scientists and Engineers*. CRC Press.
- Patterson, D. A., & Hennessy, J. L. (2017). *Computer Organization and Design: The Hardware/Software Interface* (RISC-V Edition). Morgan Kaufmann.
- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th). Wiley.